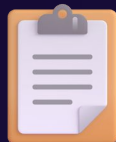


Clase 16. AI Engineer Bootcamp

Producción real

Observabilidad, seguridad y deploy —
Ramsés Camas





Agenda

Clase 16 — Producción

/01

Tracing

Logs, métricas y trazas con Phoenix

/02

Seguridad

Guardrails de input, output y tools

/03

Deploy

HF Spaces, Streamlit Cloud, Render

/04

Cierre

Checklist de release y Q&A

Recap 1-4

De llamadas sueltas a contratos confiables.



- **Clase 1** Setup profesional + cliente LLM con logging.
- **Clase 2** Tokens, contexto, costo y latencia.
- **Clase 3** Prompts versionables y evaluables.
- **Clase 4** Outputs estructurados con Pydantic.
- **Hito** core/llm_client.py, tokenlab, promptkit, contracts

Recap 5-12

De responder preguntas a sistemas con estado y razonamiento.



- **Clases 5-8** RAG serio — embeddings, ingesta, vectorstore, retrieval.
- **Clases 9-11** Cadenas, ReAct, multiagente con LangGraph.
- **Clase 12** Memoria persistente por sesión y usuario.
- **Hito** rag/, agents/, orchestration/, memory/.

Recap 13-15

De confiar a tener señal
objetiva de calidad.



- **Clase 13** MCP — integración estándar con herramientas externas.
- **Clase 14** Testing — contratos, golden tests, replay, regresión.
- **Clase 15** Evals + DSPy — métricas semánticas y optimización guiada.
- **Hito** `mcp/`, `evals/`, `dspy_lab/`

Objetivo de hoy



Llevar el DocOps Agent a una URL pública que puedan compartir hoy mismo. Instrumentarlo para saber qué pasa cuando algo falla. Protegerlo contra los tres ataques más comunes. Salir con un README que les consiga la entrevista.



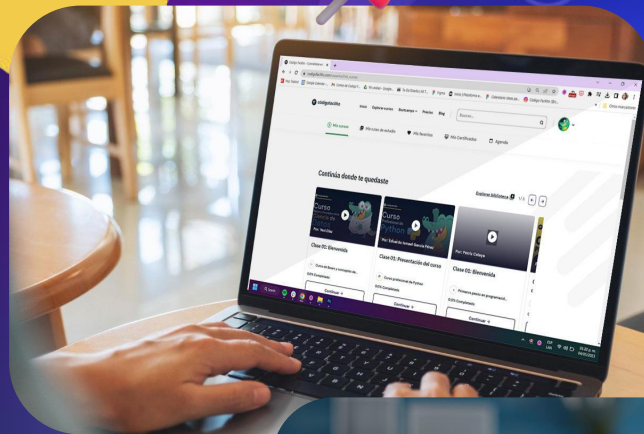
En tu laptop vs. en producción

En tu laptop: corres `python main.py` y funciona. El happy path está cubierto, los tests verdes, el demo sale redondo.

En producción: latencia variable, usuarios raros, datos sensibles, regresiones silenciosas. Sin observabilidad debuggeas a ciegas. Sin seguridad eres el titular de mañana. Sin deploy, nadie lo usa.

/01 Los tres pilares

En ese orden. Sin los dos primeros, deploy es una bomba.



Más que logs

print() y logs planos no sirven para agentes que encadenan decisiones.



- **El problema** Un agente hace 5 tool calls, 3 LLM calls y 2 retrievals por query.
- **Lo que sí sirve** Trazas jerárquicas: quién llamó a quién, con qué input y output.
- **Tres señales** Logs, métricas y trazas. Para AI systems, las trazas son la estrella.
- **Regla** Si no puedes reconstruir una query después, no tienes observabilidad.
-

OpenTelemetry + Phoenix



OpenTelemetry

Estándar abierto,
vendor-neutral. Instrumenta
LangChain y LangGraph con ~5
líneas. Exporta a cualquier
backend compatible: Phoenix,
Jaeger, Datadog.



Phoenix (Arize)

UI self-hostable, gratis,
diseñada para LLMs. Corre en
Docker local. Alternativas
comerciales: LangSmith,
Langfuse, Helicone.

```
# ops/observability.py - Instrumentar
from phoenix.otel import register
from openinference.instrumentation.langchain \
import LangChainInstrumentor

tracer_provider = register(
    project_name="docops-agent",
    endpoint="http://localhost:6006/v1/traces",
)
LangChainInstrumentor().instrument(
    tracer_provider=tracer_provider
)

# 5 líneas. LangChain queda instrumentado.
```



Ver el trace

Ejecutamos una query real al DocOps Agent y abrimos Phoenix en localhost:6006.



- **Árbol completo** Nodo supervisor → decisión → tool call → respuesta.
- **Por cada LLM call** Prompt, respuesta, tokens, latencia, costo estimado.
- **Por cada tool call** Input, output, duración, errores si los hay.
- **Momento aha** Ya no debuggeas con prints. Ves el sistema razonar.
-

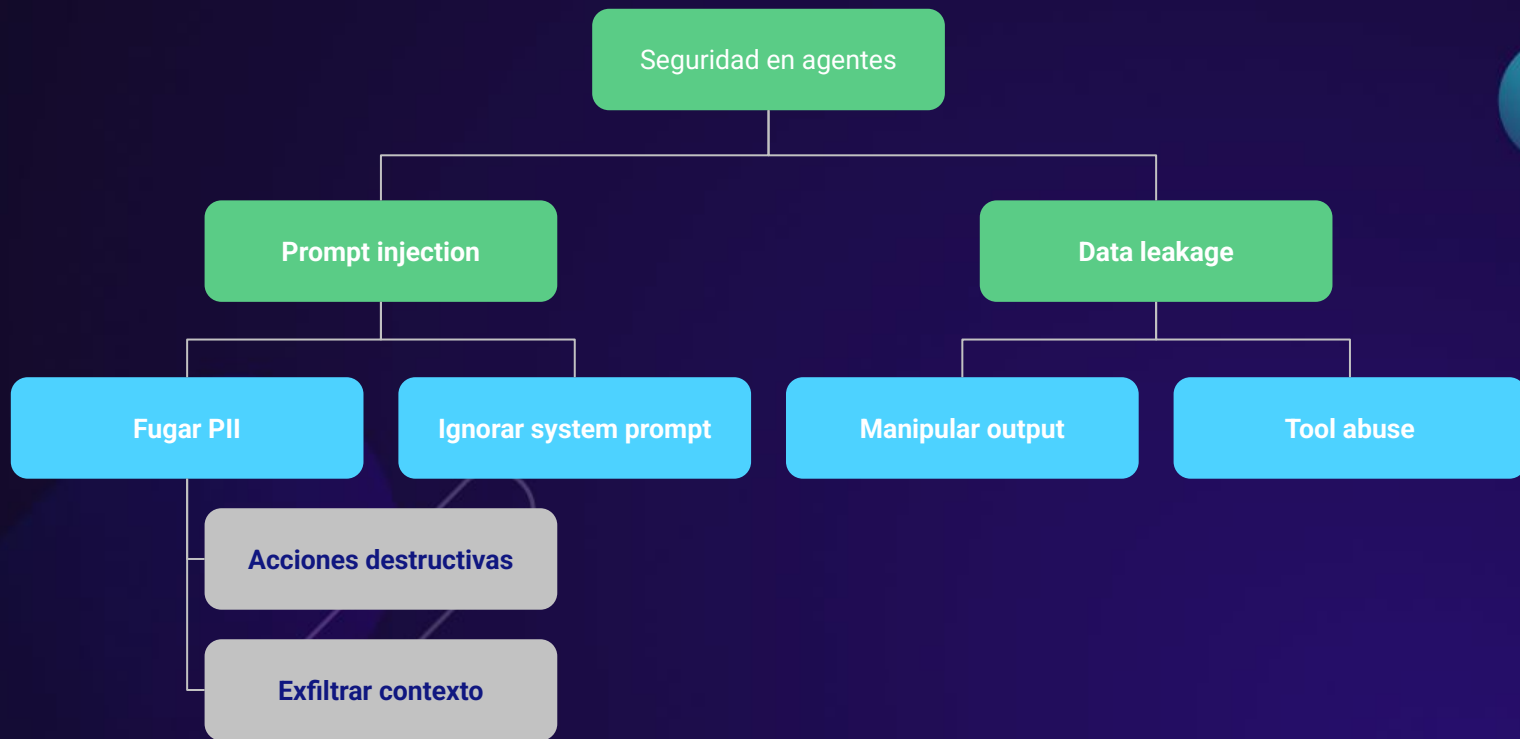
Métricas clave

No mides todo. Mide lo que afecta al usuario y a tu factura.



- **Latencia p50/p95/p99** No el promedio. Los outliers matan la UX.
- **Tokens por request** Correlación directa con tu factura mensual.
- **Costo por conversación** El único número que le importa a tu jefe.
- **Tool call success rate** Si falla al usar herramientas, el sistema es inútil.
- **Error rate por tipo** Distinguir fallos de modelo vs fallos de integración.

Las tres amenazas



Input

Primera línea de defensa:
validar lo que entra antes de
tocar el LLM.



- **Patrones conocidos** Regex y keywords para ataques frecuentes.
- **Clasificador ligero** Modelo pequeño como prompt-injection-detector.
- **Límites duros** Longitud máxima, caracteres permitidos, idioma.
- **Principio** Fallar rápido, fallar barato. No gastes tokens en veneno.
-

```
# ops/guardrails.py - Input guardrail
INJECTION_PATTERNS = [
    r"ignore previous instructions",
    r"disregard (?:all|the) (?:above|prior)",
    r"you are now",
    r"system prompt",
]

def check_input(msg: str) -> GuardrailResult:
    for p in INJECTION_PATTERNS:
        if re.search(p, msg, re.IGNORECASE):
            return GuardrailResult(
                blocked=True,
                reason="prompt_injection")
    if len(msg) > 4000:
        return GuardrailResult(
            blocked=True, reason="too_long")
    return GuardrailResult(blocked=False)
```



Output

Segunda línea: validar lo que sale antes de mostrárselo al usuario.



- **PII** Emails, teléfonos, RFC, CURP, números de tarjeta.
- **Fugas de sistema** Fragmentos del prompt base o de otros usuarios.
- **Contenido peligroso** URLs maliciosas, dominios no permitidos.
- **Herramientas** Presidio de Microsoft, guardrails-ai, o reglas propias.
-

```
# ops/guardrails.py - Output scrubber
from presidio_analyzer import AnalyzerEngine

analyzer = AnalyzerEngine()

def scrub_output(response: str) -> str:
    results = analyzer.analyze(
        text=response,
        entities=["EMAIL_ADDRESS",
                 "PHONE_NUMBER",
                 "CREDIT_CARD"],
        language="es")
    for r in sorted(results,
                    key=lambda x: x.start,
                    reverse=True):
        response = (response[:r.start]
                   + f"[{r.entity_type}]"
                   + response[r.end:])
    return response
```



Tool safety

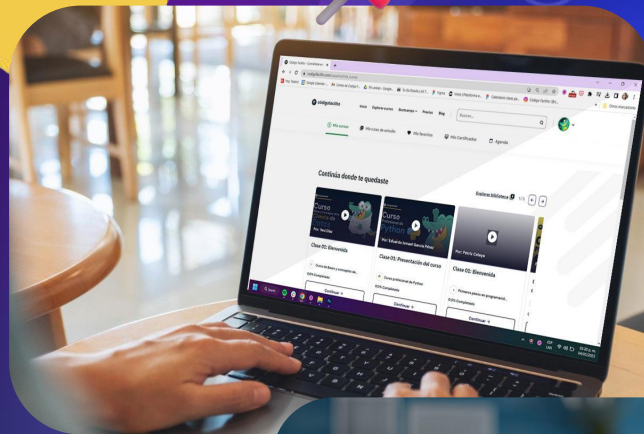
No todas las herramientas son iguales. Clasifícalas por riesgo antes de permitir las.



- **Read-only** Buscar, consultar, listar. Ejecutar libre con rate limit.
- **Write reversible** Crear borradores, staging. Log + rate limit agresivo.
- **Write destructiva** Eliminar, enviar email, cobrar. Humano aprueba.
- **LangGraph interrupt()** Primitiva nativa para human-in-the-loop.
- **Regla** Si cuesta dinero o es irreversible, humano en el loop.

/02 Tres niveles

Del demo público a la producción seria.



HF Spaces

Perfecto para portafolio. Los recruiters ya la reconocen como entorno legítimo.



- **Soporte** Docker, Gradio, Streamlit nativo. Git push para deploy.
- **URL**
huggingface.co/spaces/tu_usuario/docops-agent.
- **Free tier** 16 GB RAM, 2 vCPU. Suficiente para demo público.
- **Persistencia** Archivos del Space. Bueno para ChromaDB embebido.
- **Costo** Gratis permanente. Sin tarjeta de crédito.

```
# 1. Crear Space en huggingface.co (Docker)
git clone \
    https://huggingface.co/spaces/USER/docops-agent
cd docops-agent
```

```
# 2. Copiar tu repo + Dockerfile
cp -r ../docops-agent-local/* .
```

```
# 3. Configurar secrets en la UI web:
# Settings -> Variables and secrets
# GROQ_API_KEY, CHROMA_DB_PATH, etc.
```

```
# 4. Push y listo
git add .
git commit -m "Deploy v1"
git push
```

```
# ~3 min: URL pública para compartir hoy.
```



Streamlit

UI pulida sin Docker. Deploy directo desde GitHub.



- **Setup** Conecta GitHub, selecciona repo y branch.
- **URL** `tu-app.streamlit.app` — limpia y compatible.
- **Secrets** Pegar como TOML en la UI. Sin archivos en el repo.
- **Hot reload** Cada push a main redeploy automático.
- **Limitación** Python puro. Menos flexibilidad que Docker.

```
# Estructura mínima del repo:
# streamlit_app.py
# requirements.txt
# .streamlit/config.toml

# 1. En share.streamlit.io:
# - Login con GitHub
# - New app -> repo + branch + main file

# 2. Agregar secrets (TOML):
[secrets]
GROQ_API_KEY = "gsk_..."
CHROMA_PATH = "/mount/src/repo/data"

# 3. Deploy automático en cada push a main.
# Logs en la UI. Hot reload. Listo.
```



Render

El salto a producción de verdad. Multi-servicio con docker-compose.



- **Flexibilidad** Agente + Phoenix + Qdrant como servicios separados.
- **Infra as code** render.yaml en el repo. Reproducible.
- **Persistent disks** Health checks, env vars, custom domains.
- **Free tier** Cold start de 15 min. Aceptable para demo.
- **Paid** \$7 USD/mes por servicio. Barato para side project.

```
# render.yaml - infra as code
services:
  -type: web
    name: docops-agent
    env: docker
    dockerfilePath: ./Dockerfile
    envVars:
      - key: GROQ_API_KEY
        sync: false
    healthCheckPath: /health

  -type: pserv
    name: qdrant
    env: docker
    disk:
      name: qdrant-data
      mountPath: /qdrant/storage
      sizeGB: 1
```



Comparativa rápida

	Setup	Multi-servicio	Cold start	Mejor para
HF Spaces	Git push	No	No	Portafolio y demo pública
Streamlit Cloud	GitHub connect	No	Sí (cold)	UI rica sin infra
Render	render.yaml	Sí	Sí (free)	Producción seria

Release

Antes de compartir tu URL al mundo, verifica estos mínimos.



- **Calidad** Evals pasan, golden tests verdes, regresión ok.
- **Observabilidad** Phoenix o equivalente activo y visible.
- **Seguridad** Input + output guardrails + tool safety configurados.
- **Secretos** Fuera del repo. .env en .gitignore. Variables en la UI.
- **Docs** Health endpoint, rate limit, README de portafolio.

README

Tu README debe responder en 30 segundos qué, cómo y por qué.



- **Qué hace** Una línea, sin jerga. El recruiter no sabe qué es RAG.
- **Demo** Screenshot o GIF + URL pública funcional.
- **Arquitectura** Diagrama: RAG + agente + MCP + evals + deploy.
- **Cómo se evalúa** Métricas reales con números, no adjetivos.
- **Limitaciones** Honestidad sobre qué no funciona. Ingenieros sobre vendedores.



“16 clases. Un producto real.”

— Construyan algo que importe.